

Svelte **Review Cards**



Eight concepts an interviewer can probe and a beginner must own.
Read the question, cover the answer, answer aloud, then check the
code and the figure.

20 — flushSync: state writes are batched, DOM reads are stale

REACTIVITY DOM TIMING

Q. You set `claps = 5` and the very next line reads the DOM — why does it still show 4?

ANSWER. Because assigning state does not touch the DOM. Svelte queues the change and applies all queued updates together after your code finishes. Any DOM read between the assignment and that batch reads the previous frame. Calling `flushSync()` right after the write forces the queue to apply immediately, so the next line sees the new value.

WHY IT WORKS. The expectation feels right because JavaScript lines run in order and every `let` change you ever made was visible on the next line. But the DOM is not the variable; it is a printout of the variable, and Svelte prints on a schedule, not per keystroke. Batching is what keeps a burst of ten writes from repainting ten times.

ClapButton.svelte

```
<script>
  import { flushSync } from 'svelte';
  let claps = $state(4);
  let counterEl;

  function read() {
    claps = 5;
    flushSync(); // apply the pending DOM update now
    console.log(counterEl.textContent); // "5" —
    without flushSync: "4"
  }
</script>

<h1 bind:this={counterEl}>{claps}</h1>
<button onclick={read}>Clap and read</button>
```

DIAGRAM CONCEPT. The Press Run: writes are stories filed to the press room and printed on a schedule; `flushSync` is a special edition printed on the spot, so the next line reads fresh news instead of yesterday's paper.

THE PRESS RUN

NO FLUSHSYNC · READS THE OLD EDITION



FLUSHSYNC() · SPECIAL EDITION PRINTS NOW



- ① **Summary.** A state write joins a queue; the DOM updates when the batch prints. Reach for `flushSync()` only when the very next line must read fresh DOM — everywhere else, let the press run on schedule.

21 — Await blocks: the promise's whole life, in markup

ASYNC TEMPLATE PROMISES

Q. The article fetch is still in flight — where do the spinner and the error message go, without a loading boolean and a try/catch in script?

ANSWER. Inside the `{#await}` block. It owns the promise's entire life in the template: the pending branch renders while the fetch is in flight, then exactly one of `{:then}` (the story landed) or `{:catch}` (it failed) replaces it. No booleans, no script glue, no manual state machine.

WHY IT WORKS. The instinct to hand-roll comes from plain JavaScript, where promise states are handled in `.then` and `.catch` callbacks, so the UI for them must be driven from script too. In Svelte the template can await directly, so the three states of the promise become three named places in the markup — the spinner has a home and the error has a home, and Svelte swaps between them for you.

ArticlePage.svelte

```
<script>
  import { fetchArticle } from './api.js';
  let { id } = $props();
</script>

{#await fetchArticle(id)}
  <p>Loading the story...</p>           <!-- pending:
in flight -->
{:then article}
  <ArticleView {article} />           <!-- resolved
-->
{:catch err}
  <p>Could not load: {err.message}</p> <!-- rejected
-->
{/await}
```

DIAGRAM CONCEPT. Waiting on the Wire: the newsroom waits for the story at the pending desk; when the wire arrives it lands exactly once — on the editor's desk with

the story, or at the correction desk with the error.

WAITING ON THE WIRE

1 · PENDING DESK

fetch in flight

THE SPINNER LIVES HERE



THE WIRE ARRIVES ONCE · ONE DOOR ONLY

2a · THEN

story landed

ARTICLEVIEW RENDERS

2b · CATCH

correction desk

ERROR MESSAGE RENDERS

- ① **Summary.** One block, three homes: pending holds the spinner, then holds the story, catch holds the error. The promise resolves once, Svelte swaps the branch, and your script stays free of loading booleans.